

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 3
з дисципліни «Основи програмування»
«Мова програмування Сі. Компіляція, функції вводу/виводу»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи

Вивчити поняття: компіляції, функцій вводу/виводу.

1. Завдання 1. Ілюстрація специфікаторів printf()

Програма демонструє вивід одного й того ж цілого числа 42 у десятковому, двійковому та шістнадцятковому вигляді, дійсного числа 123.456 у формах %f, %e та %g, а також символу, рядка і вказівника.

2.1. Текст програми (task1.c)

```
#include <stdio.h>

int main() {
    int num = 42;
    printf("Десяткове: %d\n", num);
    printf("Двійкове: %b\n", num);
    printf("Шістнадцяткове: %x\n", num);
    double value = 123.456;
    printf("Плаваюче: %f\n", value);
    printf("Експоненційне: %e\n", value);
    printf("Гнучка: %g\n", value);
    char c = 'A';
    printf("Символ: %c\n", c);
    char str[] = "Hello";
    printf("Стрічка: %s\n", str);
    int n = 42;
    int *p = &n;
    printf("Вказівник: %p\n", p);
    return 0;
}
```

Рис. 1. Початковий код програми task1.c

2.2. Результат виконання

```
➤ L>> ./task1
Десяткове: 42
Двійкове: 101010
Шістнадцяткове: 2a
Плаваюче: 123.456000
Експоненційне: 1.234560e+02
Гнучка: 123.456
Символ: A
Стрічка: Hello
Вказівник: 0x7fff2e5920b8
```

Рис. 2. Вивід програми task1

%d дає звичайне десяткове 42, %b (GNU) — 101010, %x — 2a. Для дійсного числа %f показує фіксовану кількість знаків після коми, %e — експоненційну форму 1.234560e+02, а %g обирає найкомпактніший варіант. Символ, рядок і адреса вказівника виводяться відповідно специфікаторами %c, %s і %p.

1. Завдання 2. Форматований ввід і таблиця

Друга програма зчитує прізвище з ініціалами, електронну пошту та улюблений колір, після чого виводить їх у вигляді простої таблиці з вирівняними колонками.

3.1. Текст програми (task2.c)

```
Martsinko > Lab3 > task2.c
1  #include <stdio.h>
2
3  int main()
4  {
5      char name[50];
6      char email[50];
7      char color[30];
8
9      printf("Прізвище та ініціали: ");
10     scanf(" %49[^\n]", name);
11
12     printf("Email: ");
13     scanf("%49s", email);
14
15     printf("Улюблений колір: ");
16     scanf("%29s", color);
17
18     printf("%-1s %-20s %-32s %-15s\n",
19           "№", "Прізвище, Ініціали", "Ел. пошта", "Колір");
20     printf("-----\n");
21     printf("%-1d %-28s %-25s %-15s\n",
22           1, name, email, color);
23     return 0;
24 }
```

Рис. 3. Початковий код програми task2.c

Для прізвища використано `scanf(" %49[^\n]", name)` — це дозволяє зчитати рядок з пробілами до кінця рядка і не вийти за межі буфера. Для email і кольору достатньо `%49s` / `%29s`. Заголовок і рядок даних друкуються з фіксованими ширинами полів (`%-5s`, `%-25s`, `%-30s`, `%-15s`), тому колонки вирівнюються.

3.2. Результат виконання

```
└─>> ./task2
arch in ../2026_OP_FEP_13/Martsinko/Lab3
└─>> ./task2
Прізвище та ініціали: марцінко н р
Email: nazar1404m@gmail.com
Улюблений колір: чорний
№ Прізвище, Ініціали Ел. пошта Колір
-----
1 марцінко н р nazar1404m@gmail.com чорний
```

Рис. 4. Ввід даних і сформована таблиця

Після введення «Марцінко Н Р», `nazar1404m@gmail.com` і «чорний» програма виводить таблицю з одним рядком даних. Ширини полів підібрані так, щоб заголовок і значення стояли один під одним.

4. Алгоритм роботи програм

Завдання 1:

1. Оголосити змінні потрібних типів (int, double, char, char[], int*).
2. Присвоїти їм тестові значення.
3. Послідовно вивести кожну змінну з відповідним специфікатором printf.
4. Завершити програму.

Завдання 2:

1. Оголосити масиви для імені, email і кольору.
2. Зчитати прізвище з ініціалами (з урахуванням пробілів).
3. Зчитати email.
4. Зчитати улюблений колір.
5. Вивести заголовок таблиці з фіксованими ширинами колонок.
6. Вивести розділову лінію.
7. Вивести рядок даних у тих самих ширинах.
8. Завершити програму.

Висновок

У ході виконання лабораторної роботи було досліджено роботу функцій форматowanego вводу/виводу мовою C та особливості їх практичного застосування. Головним висновком цієї роботи стало усвідомлення того, що стандартні механізми scanf() та printf() є чи не найбільш незручними та небезпечними інструментами через архаїчну систему модифікаторів.

Необхідність вручну підбирати специфікатори на кшталт scanf(" %49[^\n]", name) лише для того, щоб банально прочитати рядок із пробілом, у поєднанні з обов'язковим ручним контролем довжини буферів на зразок %49s чи %29s заради захисту від переповнення пам'яті, створює колосальні незручності на рівному місці. Будь-яка помилка в символі формату чи його невідповідність типу змінної ламає або логіку роботи, або пам'ять. Зі свого боку printf() перетворює навіть

вирівнювання простої таблиці на ручний підбір модифікаторів ширини та відступів типу %-28s, де найменша неточність призводить до зміщення всіх колонок.